Monte Carlo Methods

1 Introduction

Monte Carlo (MC) methods are a simple way to learn from experience in reinforcement learning. Instead of using a model, they learn directly from complete episodes.

2 Conceptual Explanation

2.1 Monte Carlo Prediction

Monte Carlo prediction estimates how good a state is under a fixed policy by averaging returns from multiple episodes.

The return is:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$$

For Blackjack, $\gamma = 1$, so the return is simply the final reward.

There are two types:

- First-visit: update only first time a state appears
- Every-visit: update every occurrence

We use **first-visit MC**.

2.2 Monte Carlo Action-Value Estimation

To improve decisions, we estimate $Q(s, a)$ instead of $V(s)$.

This tells us which action is better in each state.

2.3 Backup Diagram (MC)

$$\begin{array}{c} \text{State } S_0 \to S_1 \to S_2 \to \ldots \to \text{Terminal} \\ \downarrow \\ G \text{ (final reward used for all updates)} \end{array}$$

After the episode ends, we go backward and update all visited states using the same return.

2.4 Monte Carlo Control

MC control improves policy by repeating:

- Evaluate policy (estimate Q)
- Improve policy (choose best action)

2.4.1 MC-ES (Exploring Starts)

Exploring starts ensures all state-action pairs are visited by choosing random starting actions.

Policy update:

$$\pi(s) = \arg\max_a Q(s, a)$$

but, this is not practical because in real systems we cannot force starting states.

2.5 MC Control without Exploring Starts

We use ϵ-greedy:

- Choose best action with probability $1 - \epsilon$
- Choose random action with probability ϵ

This ensures exploration.

2.6 Off-Policy MC Prediction

Derivation (simple idea):

We want expected return under π, but data comes from b.

So we correct it using probability ratios:

$$\text{Expected value under } \pi = \rho \times \text{Return from } b$$

$$\rho_{t:T} = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}$$

This ratio corrects the difference between behaviour policy and target policy by re-weighting the returns. Weighted importance sampling reduces variance.

3 Mathematical Formulation

- Return:
$$G_t = \sum \gamma^k R$$
- Value update:
$$V(s) \leftarrow V(s) + \frac{1}{N(s)}(G - V(s))$$
- Action-value:
$$Q(s, a) \leftarrow Q(s, a) + \frac{1}{N(s, a)}(G - Q(s, a))$$
- Policy:
$$\pi(s) = \arg\max_a Q(s, a)$$

4 Implementation

We implemented Blackjack with:

- State: (player sum, dealer card, usable ace)
- Actions: hit or stick
- Rewards: +1, 0, -1

Algorithms implemented:

- MC Prediction
- MC-ES Control
- ϵ-greedy Control
- Off-policy MC

5 Results

5.1 Experiment 1: MC Prediction

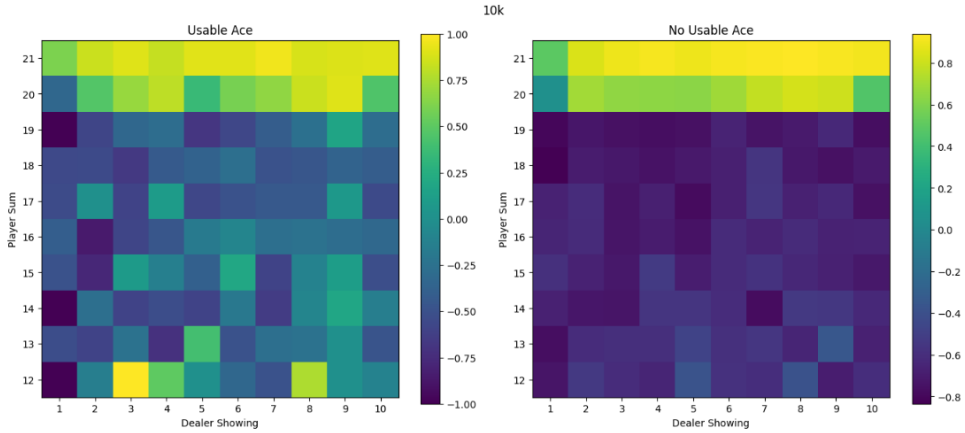


Figure 1: Value function after 10,000 episodes

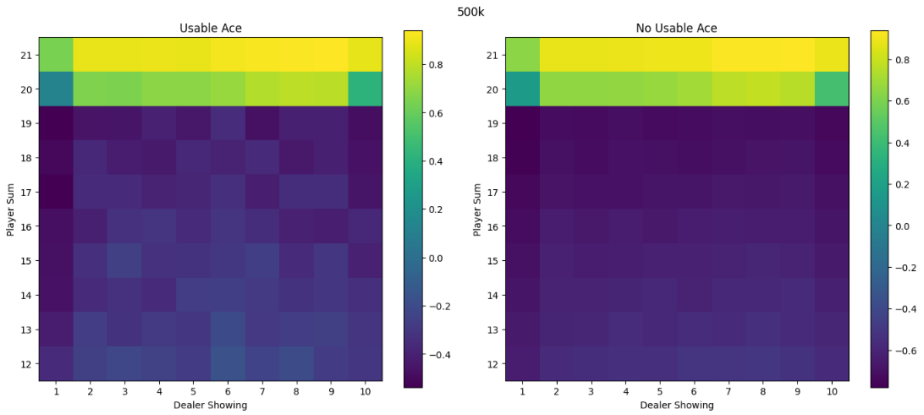


Figure 2: Value function after 500,000 episodes

At 10k episodes, the heatmaps are noisy. At 500k episodes, they become smooth and stable. This shows MC improves with more data.

5.2 Experiment 2: Policy Comparison

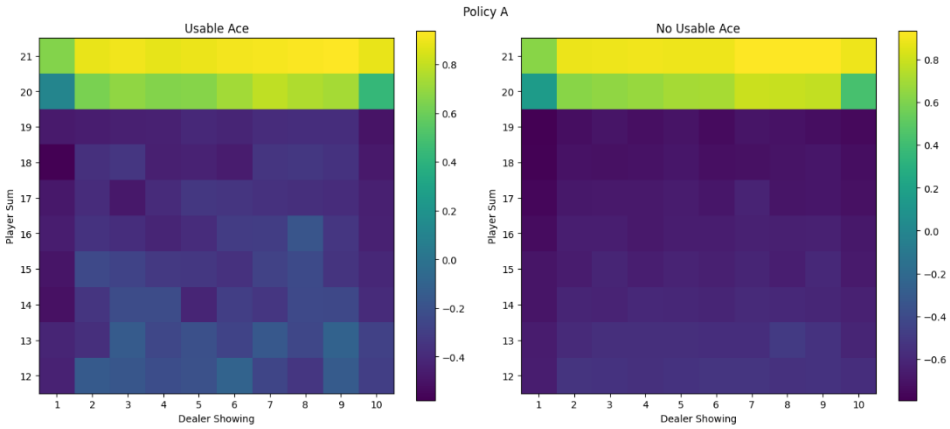
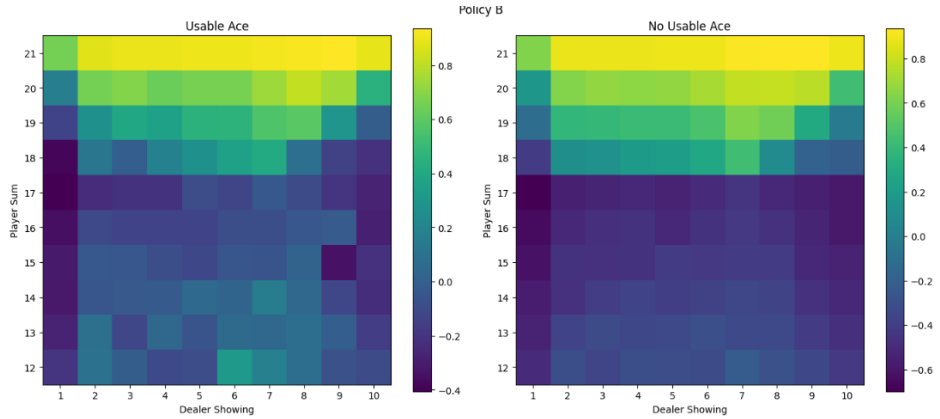
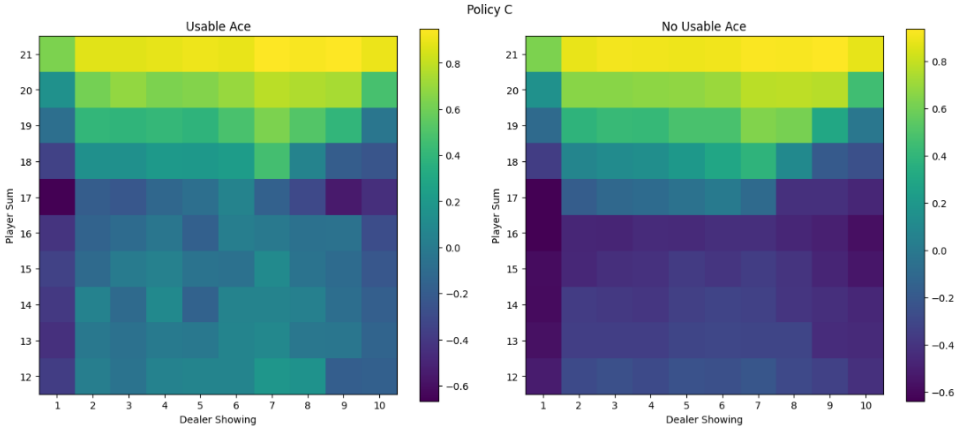


Figure 3: Policy A



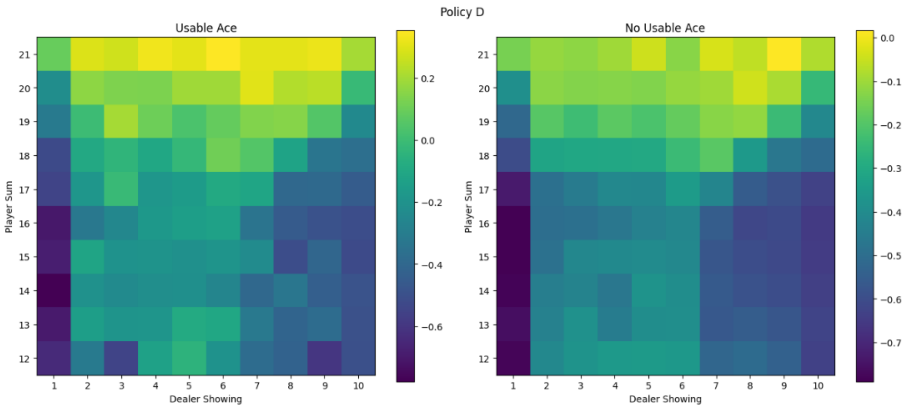
Policy B: Average Reward per Episode = -0.1132

Figure 4: Policy B



Policy C: Average Reward per Episode = -0.0801

Figure 5: Policy C



Policy D: Average Reward per Episode = -0.3949

Figure 6: Policy D

Results:

- Policy A performs worst (too conservative)
- Policy B improves performance
- Policy C performs best
- Policy D is unstable (random)

Policy C performs best because it avoids stopping too early while still reducing the chance of busting, especially against stronger dealer cards.

5.3 Experiment 3: MC-ES

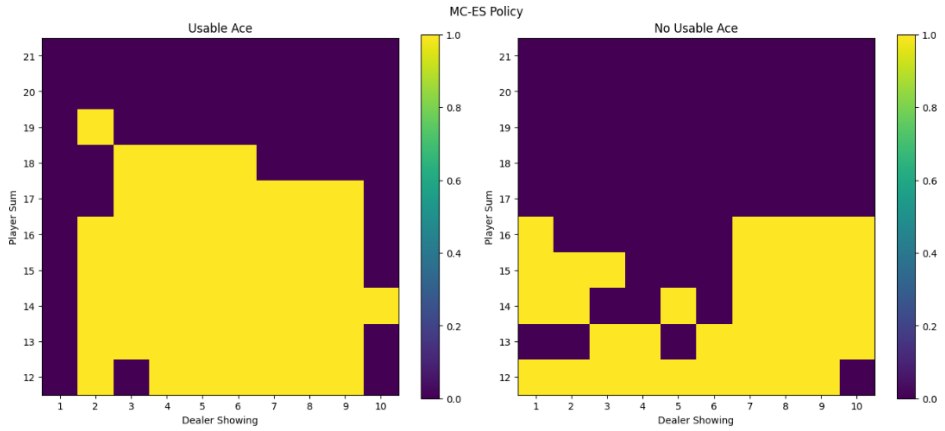


Figure 7: MC-ES Learned Policy

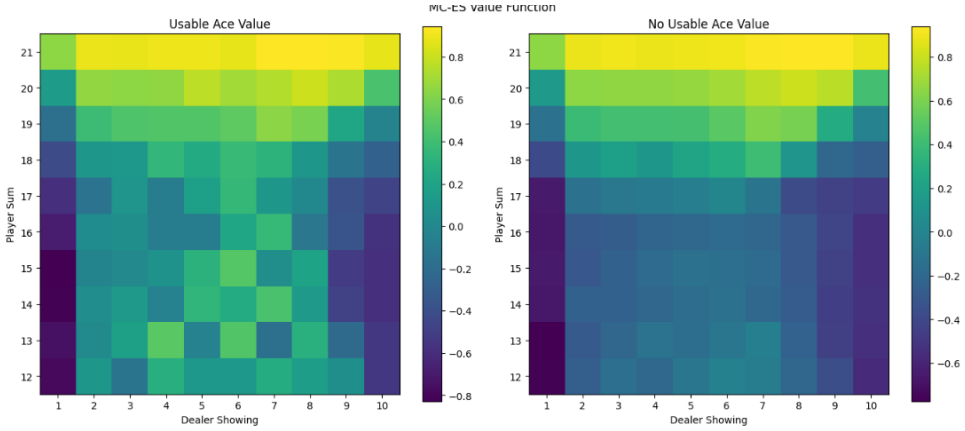


Figure 8: MC-ES Value Function

The learned policy is better than fixed policies because it adapts based on experience.

5.4 Experiment 4: ϵ-Greedy

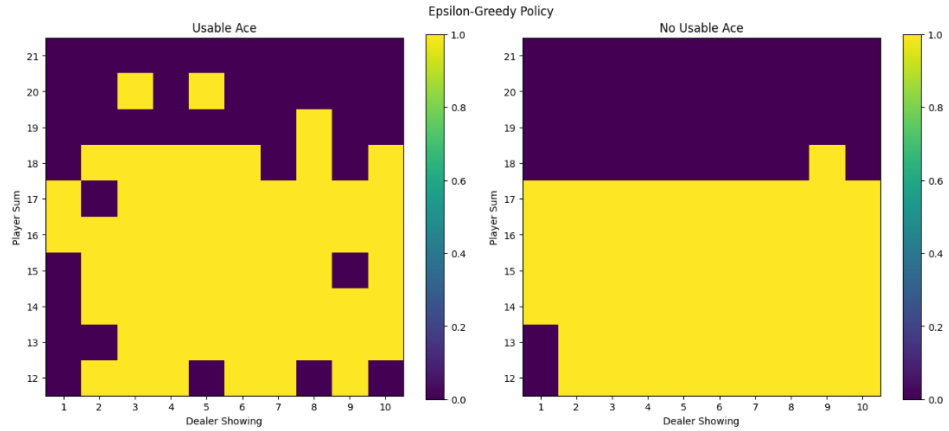


Figure 9: ϵ-Greedy Policy

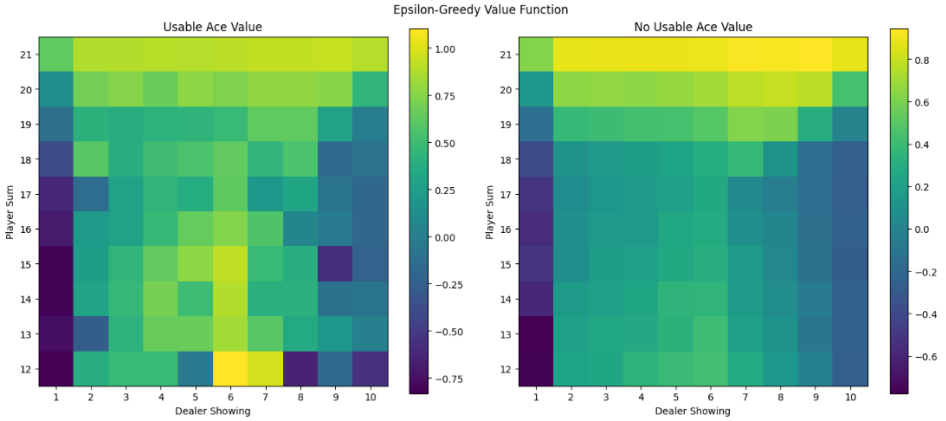


Figure 10: ϵ-Greedy Value Function

The policy is similar to MC-ES but slightly noisy due to exploration.

5.5 Off-Policy

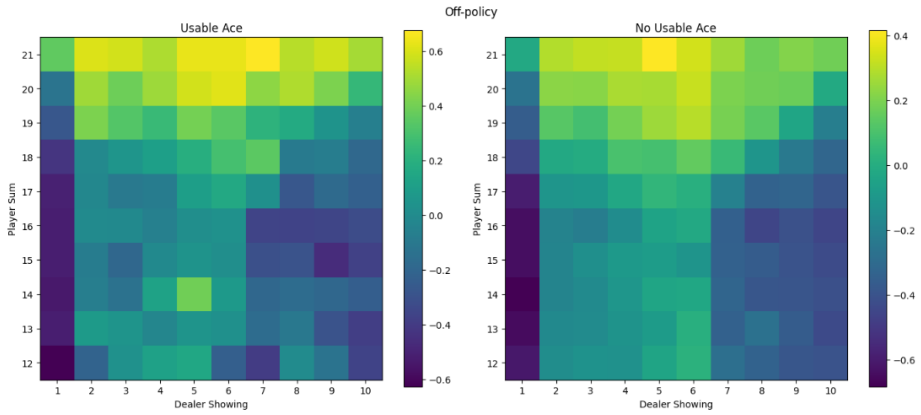


Figure 11: Off-policy Value Function

The estimates are correct but slightly noisy due to high variance.

6 Discussion

- More episodes give better estimates
- MC-ES learns near-optimal policy
- ϵ-greedy is practical and effective
- Off-policy has higher variance

Advantages:

- No model required
- Easy to understand

Limitations:

- Needs full episodes
- Can be slow
- High variance